

คู่มือผู้ดูแลระบบ

สถาปัตยกรรม การปล่อยเวอร์ชัน การดูแลฐานข้อมูล การสำรองข้อมูล และวิธีรับมือเมื่อระบบมีปัญหา

สำหรับผู้พัฒนาและผู้ดูแลระบบเท่านั้น

เอกสารนี้มีข้อมูลเชิงเทคนิคที่ใช้เข้าถึงระบบได้ ห้ามเผยแพร่ทั่วไป

ฉบับปรับปรุง สิงหาคม 2569

สารบัญ

1	ระบบประกอบด้วยอะไร และแต่ละส่วนอยู่ที่ไหน	3
2	หลักการ 0 บาท — สิ่งที่ไม่ห้ามทำลาย	4
3	ตั้งเครื่องสำหรับพัฒนา	5
4	ปล่อยเวอร์ชันใหม่ และย้อนกลับ	7
5	ฐานข้อมูล — โครงสร้างและกฎเหล็ก	9
6	ระบบแคช — วิธีการทำงานและวิธีดีบั๊ก	11
7	Rich Menu และระบบแจ้งเตือนด้วยจุดแดง	12
8	ความปลอดภัย — ปิดแล้วอะไร ยังเปิดอยู่อะไร	13
9	สำรองข้อมูล และกู้คืนลงเครื่อง	15
10	รับมือเหตุขัดข้อง	17
11	โควตาและข้อจำกัดที่ต้องจำ	18

ระบบประกอบด้วยอะไร และแต่ละส่วนอยู่ที่ไหน

ระบบนี้มีโค้ดอยู่ 3 ที่ที่เวอร์ชันไม่ผูกกันเอง เข้าใจข้อนี้ก่อนอย่างอื่น เพราะเป็นที่มาของความสับสนเกือบทั้งหมดในการดูแลระบบ

ส่วน	อยู่ที่ไหน	อัปเดตอย่างไร
หน้าเว็บในไลน์ (LIFF)	ไฟล์ *.html , app.js , app.css , config.js เผยแพร่ที่ manhaqq.github.io/koh-hr-liff/	<code>git push</code> แล้ว GitHub Pages deploy เองใน 1–2 นาที
หลังบ้าน (Apps Script)	โฟลเดอร์ <code>apps-script/</code> รันเป็น Web App ที่ URL ลงท้าย <code>/exec</code>	<code>clasp push</code> แล้วต้องสร้างเวอร์ชันและผูก deployment อีกชั้น
ฐานข้อมูล	Google Sheet ไฟล์เดียว 15 แท็บ	แก้ไขโดยตรง — ไม่ได้อยู่ใน git

จุดที่พลาดกันบ่อยที่สุด

`clasp push` ไม่ได้ ทำให้ของจริงเปลี่ยน มันแค่อัปเดตโค้ดขึ้นไปเก็บไว้ Web App ที่พนักงานใช้ยังรันเวอร์ชันเดิมอยู่ จนกว่าจะสร้างเวอร์ชันใหม่แล้วชี้ deployment ไปหา มัน ซึ่ง `scripts/release.sh` ทำให้ครบทุกขั้นแล้ว

ทำไม URL ของ deployment ห้ามเปลี่ยน

URL `/exec` ถูกฝังไว้ 2 ที่ที่เก๋ยาก คือใน `config.js` ซึ่งเสิร์ฟให้ทุกเบราว์เซอร์ และในช่อง Webhook URL ของ LINE Developers Console ถ้าสร้าง deployment ใหม่ URL จะเปลี่ยน แล้วทั้งเว็บและ webhook จะพังพร้อมกัน

วิธีที่ถูกต้องคือ ชี้ deployment เดิมไปยังเวอร์ชันใหม่ รหัส deployment ที่ใช้งานจริงถูกเขียนตายตัวไว้ในตัวแปร `LIVE_DEPLOYMENT` ทั้งใน `scripts/release.sh` และ `scripts/rollback.sh`

หลักการ 0 บาท — สิ่งห้ามทำลาย

แพ็คเกจฟรีของ LINE ส่งข้อความ push ได้ **300 ข้อความต่อเดือน** พนักงาน 60 คนหมายความว่าแจ้งเตือนได้เพียง **5 ครั้งต่อเดือน** เท่านั้น ระบบทั้งหมดจึงออกแบบมาเพื่อ**ไม่ใช่ push** เลย

แทนที่จะ	ระบบทำแบบนี้	ต้นทุน
ส่งข้อความว่ามีประกาศใหม่	สลับภาพ Rich Menu ให้ขึ้นจุดแดง	ฟรี ไม่จำกัด
ส่งข้อความเตือนเข้ากะ	พนักงานกดเพิ่มลงปฏิทิน มือถือเตือนเอง	ฟรี
ส่งข้อความบอก HR ว่ามีเรื่องใหม่	ส่งอีเมล	ฟรี
ตอบข้อความในแชท	ใช้ reply token ไม่ใช่ push	ฟรี

ก่อนเพิ่มฟีเจอร์ใด ๆ ให้ถามข้อนี้เสมอ

ฟีเจอร์นี้ต้องส่งข้อความ push หรือเปล่า ถ้าใช่ **แปลว่าคิดผิดทาง** ให้กลับไปดูว่าจะใช้จุดแดง อีเมล หรือ reply token แทนได้ไหม การเพิ่ม push เพียงจุดเดียวจะทำให้โควตาหมดภายในไม่กี่วัน แล้วระบบทั้งระบบจะเจียบไปทั้งเดือนโดยไม่มีใครรู้ว่าเพราะอะไร

โควตา 300 ข้อความถูกเก็บไว้ทั้งก่อนสำหรับเหตุผลจริง ๆ ผ่านเมนู **ส่งด่วนเข้าแชททุกคน** เท่านั้น

ตั้งเครื่องสำหรับพัฒนา

```
git clone https://github.com/manhaqq/koh-hr-liff.git
cd koh-hr-liff

# เครื่องมือ sync โค้ดหลังบ้าน
npm install -g @google/clasp
clasp login          # ใช้บัญชี Google ที่เป็นเจ้าของ Apps Script

# สร้างภาพ Rich Menu
python3 -m pip install --user Pillow

# สร้างคู่มือ PDF (ไม่จำเป็นถ้าไม่แก้คู่มือ)
brew install pango gdk-pixbuf libffi python@3.13
/opt/homebrew/bin/python3.13 -m venv .venv-pdf
.venv-pdf/bin/pip install weasyprint
```

ต้องเปิด Apps Script API ก่อนใช้ clasp push

เข้า script.google.com/home/usersettings แล้วเปิดสวิตช์ **Google Apps Script API** ถ้าไม่เปิด clasp push จะขึ้น error ว่า User has not enabled the Apps Script API ส่วนคำสั่งอ่านอย่าง clasp versions ยังทำงานได้ตามปกติ ซึ่งทำให้เข้าใจผิดว่าตั้งค่าครบแล้ว

คำสั่งที่ใช้บ่อย

คำสั่ง	ทำอะไร
<code>cd apps-script && clasp pull</code>	ดึงโค้ดหลังบ้านล่าสุดลงมา เพื่อมีคนแก้ผ่านหน้าเว็บ
<code>cd apps-script && clasp push</code>	อัปโหลดโค้ดขึ้นไป (ยังไม่มีผลกับของจริง)
<code>cd apps-script && clasp open-script</code>	เปิดหน้า Apps Script editor
<code>python3 richmenu/build.py</code>	สร้างภาพเมนูทั้ง 5 แบบใหม่
<code>./scripts/release.sh v0.x.0 "คำอธิบาย"</code>	ปล่อยเวอร์ชันใหม่ทั้งระบบ
<code>./scripts/rollback.sh</code>	ดูเวอร์ชันที่ย้อนได้

Token และรหัสลับไม่ได้อยู่ในโค้ด

ทั้งหมดอยู่ใน Script Properties ของ Apps Script (เปิด editor แล้วไปที่  Project Settings → Script Properties) จึง commit โค้ดขึ้น GitHub แบบสาธารณะได้อย่างปลอดภัย **ห้ามเผยแพร่ token ลงในไฟล์โค้ดเด็ดขาด**

ปล่อยเวอร์ชันใหม่ และย้อนกลับ

ปล่อยเวอร์ชันใหม่

```
./scripts/release.sh v0.4.0 "เพิ่มระบบข้อสอบ"
```

สคริปต์ทำ 5 ขั้นตอนตามลำดับนี้

- 1 สร้างภาพ Rich Menu ใหม่จากไฟล์ออกแบบ แล้ว commit ถ้าภาพเปลี่ยน
- 2 `clasp push` อัปเดตโค้ดหลังบ้าน
- 3 `clasp version` สร้างเวอร์ชันถาวรที่ย้อนกลับมาได้
- 4 ชี้ deployment เติมนิยามเวอร์ชันใหม่ — **URL ไม่เปลี่ยน**
- 5 ติด git tag แล้ว push ขึ้น GitHub ซึ่งจะ deploy หน้าเว็บเอง

หลังปล่อยเวอร์ชัน ต้องทำมืออีก 2 อย่าง

1. ถ้าภาพ Rich Menu เปลี่ยน ให้เปิดชีตแล้วกดเมนู **Rich Menu → สร้าง/อัปเดต Rich Menu ทั้งหมด**
2. ถ้ามีทริกเกอร์หรือ oauth scope ใหม่ ให้กดเมนู **ติดตั้งงานอัตโนมัติ** และกด Authorize ใหม่ ทำตอนร้านไม่ยุ่ง เพราะระหว่างที่ยังไม่ authorize ระบบจะหยุดทำงาน

ย้อนกลับเวอร์ชันเก่า

```
./scripts/rollback.sh          # ดูรายการเวอร์ชัน
./scripts/rollback.sh v0.3.0   # ย้อนกลับ
```

ฝั่งหลังบ้านย้อนได้ทันทีโดยไม่ต้องแตะโค้ดเลย เพราะ Apps Script เก็บทุกเวอร์ชันไว้ถาวร แค่ชี้ deployment เติมนิยามกลับไปเวอร์ชันเก่า ใช้เวลาไม่กี่วินาที และ URL ไม่เปลี่ยน พนักงานไม่รู้สึกรว่าอะไรจะสะดุด

ฝั่งหน้าเว็บใช้ `git revert` ไม่ใช่ `git checkout` แล้ว force push เพราะ force push จาก detached HEAD จะลบ commit ทั้งหมดหลังแท็กนั้นทิ้งถาวร ซึ่งเป็นการทำลายข้อมูล ไม่ใช่การย้อนกลับ ส่วน revert สร้าง commit ใหม่ที่ย้อนการเปลี่ยนแปลง ประวัติยังอยู่ครบและย้อนซ้ำได้อีก

สองอย่างที่สคริปต์ย้อนให้ไม่ได้

ข้อมูลและโครงสร้างในซีต — ถ้าเวอร์ชันที่แล้วเพิ่มหรือย้ายคอลัมน์ ต้องกู้เองจาก **ไฟล์ → ประวัติเวอร์ชัน** ในซีต หรือจากไฟล์สำรองรายสัปดาห์

ภาพ Rich Menu — ต้องกดเมนูสร้างใหม่เองในซีต

แปลว่าการเปลี่ยนแปลงรอบนั้นแต่ละโครงสร้างซีต **ต้องสำรองซีตก่อนปล่อยเวอร์ชันเสมอ**

ฐานข้อมูล — โครงสร้างและกฎเหล็ก

เก็บทั้งหมด

กลุ่ม	เก็บ
คนกรอกเอง	Employees OrgChart Announcements Handbook AppGuide ShiftPattern Schedule Shifts Reports FAQ Settings
ระบบเขียนเอง (ห้ามลบคอลัมน์)	Tickets Leave AuditLog BroadcastLog

กฎเหล็ก 6 ข้อ

- ❶ ห้ามเปลี่ยนชื่อคอลัมน์แถวที่ 1 — โค้ดอ่านชื่อคอลัมน์ตรง ๆ เปลี่ยนเมื่อไหร่ไฟล์นั้นจะกลายเป็นคำว่างแบบเจียบ ๆ ระบบยังทำงานต่อได้แต่ข้อมูลหายหมด ซึ่งหาสาเหตุยากมาก
- ❷ ห้ามลบแถวพนักงานที่ลาออก — กฎหมายแรงงานไทยให้เก็บทะเบียนลูกจ้างอย่างน้อย 2 ปี ให้ใช้เมฆตัดสิทธิ์แทน
- ❸ ห้ามใส่เลขบัตรประชาชน — เก็บได้แค่เบอร์โทร 4 ตัวท้ายในคอลัมน์ `phoneLast4`
- ❹ ห้ามใส่เงินเดือน — ให้แยกไฟล์ที่จำกัดสิทธิ์ต่างหาก
- ❺ ห้ามแชร์แบบ "ทุกคนที่มีลิงก์" — เฉพาะ HR และผู้บริหาร
- ❻ `lineUserId` ห้ามออกนอกไฟล์นี้ — เป็นข้อมูลส่วนบุคคลตาม PDPA ห้ามส่งกลับใน API ห้ามแสดงบนหน้าจอ ห้ามเขียนลง log ห้ามใส่ในข้อความ error

เพิ่มคอลัมน์หรือเก็บใหม่

เพิ่มได้ แต่ต่อท้ายเท่านั้น ห้ามแทรกตรงกลางและห้ามสลับลำดับ และให้เขียนฟังก์ชัน `ensureXxxSheets()` ของตัวเองที่สร้างเก็บเฉพาะเมื่อยังไม่มี

อย่าใช้ `initDatabase()` เพื่อเพิ่มคอลัมน์

`initDatabase()` เขียนกับแถวที่ 1 ถ้าโครงสร้างจริงไม่ตรงกับที่โค้ดคาดไว้ หัวคอลัมน์อาจถูกเขียนผิดตำแหน่งแล้วข้อมูลทั้งเก็บจะอ่านผิดทั้งหมด ต้องสำรองซีตทั้งไฟล์ก่อนรันเสมอ

เรื่องวันที่ที่ต้องระวัง

Google Sheets มักแปลงข้อความ `2026-08-25` เป็นวันที่จริงโดยอัตโนมัติ แล้วแสดงตามรูปแบบภาษาของไฟล์ เช่น `25/8/2026` ซึ่งจะทำให้การเปรียบเทียบวันที่พังทั้งระบบ ฟังก์ชัน `cellToString_()` ใน `02_Data.js` จึงบังคับให้ทุกวันที่กลับมาเป็น `yyyy-MM-dd` และเวลาเป็น `HH:mm` เสมอ **อย่าถอดออก**

ระบบแคช — วิธีการทำงานและวิธีดีบั๊ก

เดิม `readTable()` อ่านทั้งแค็บใหม่ทุกครั้งที่ถูกเรียก และการเปิดหน้าเดียวอาจอ่านแค็บเดิมซ้ำหลายรอบ ซึ่งเป็นสาเหตุหลักที่ระบบช้า ตอนนี้มีแคช 2 ชั้นทับอยู่

ชั้น	ขอบเขต	อายุ
memo ในหน่วยความจำ	กั้นการอ่านซ้ำภายในการทำงานรอบเดียว	จบเมื่อ request จบ
CacheService	กั้นการอ่านซ้ำข้ามคำขอ แบ่งเก็บเป็นชั้น ๆ	5 นาที

แคชถูกล้างเมื่อไหร่

คีย์ของแคชผูกกับ "เลขเวอร์ชันของตาราง" การล้างจึงทำโดยเปลี่ยนเลขนั้น ซึ่งเกิดได้ 3 ทาง

- 1 **ตอนโค้ดเขียนข้อมูล** — `appendRow()` และ `updateRow()` เรียก `bumpTableVersion_()` ให้เอง
- 2 **ตอนคนแก้ไขด้วยมือ** — ทริกเกอร์ `onEditInvalidateCache` ★ ข้อนี้สำคัญที่สุด เพราะ HR แก้ไขโดยตรง เป็นเรื่องปกติของระบบนี้ ซึ่งไม่ผ่านโค้ดฝั่งเขียนเลย
- 3 **อายุหมดเอง** — กั้นพลาดถ้าสองข้อบนไม่ทำงาน

อาการ "แก้ไขแล้ว แต่หน้าเว็บยังเป็นข้อมูลเก่า"

เกือบทุกครั้งแปลว่าทริกเกอร์ `onEditInvalidateCache` ไม่ได้ติดตั้ง ให้เปิดชิตแล้วกด  ติดตั้งงานอัตโนมัติ

ใหม่ หรือถ้าต้องการล้างทันทีทั้งหมด ให้รันฟังก์ชัน `clearAllTableCache()` ใน editor

ถ้าอยากอ่านข้ามแคชในโค้ด ใช้ `readTable(ชื่อแค็บ, true)`

ทำไม `updateRow` ยังเขียนทีละเซลล์

ดูเหมือนช้าและน่าจะรวบเป็นครั้งเดียวได้ แต่ทำไม่ได้โดยตั้งใจ เพราะบางแค็บมีคอลัมน์สรุปที่เป็นสูตร ถ้าอ่านทั้งแถว แล้วเขียนกลับด้วย `setValues` สูตรจะถูกทับด้วยค่าคงที่และหายไปถาวร

Rich Menu และระบบแจ้งเตือนด้วยจุดแดง

ระบบใช้เมนู 5 ชุด การสลับระหว่างชุดคือ "ระบบแจ้งเตือน" ทั้งหมดของเรา

ชุด	ใช้กับใคร
guest	คนที่ยังไม่ยืนยันตัวตน เห็นแค่ปุ่มยืนยันตัวตน — เป็นด่านกันคนนอก
main	พนักงานปกติ ไม่มีอะไรใหม่
main-n	มีประกาศใหม่ — จุดแดงที่ปุ่มประกาศ
main-h	HR ตอบเรื่องแล้ว — จุดแดงที่การ์ดติดต่อ HR
main-nh	มีทั้งสองอย่าง

เปลี่ยนภาพเมนู

```
cp ~/Downloads/เมนูใหม่.png richmenu/src/base.png # ต้องเป็น 2500 x 1686
python3 richmenu/build.py
git add richmenu && git commit -m "เมนูใหม่" && git push
```

แล้วเปิดชัตกดเมนู  Rich Menu → สร้าง/อัปเดต Rich Menu ทั้งหมด

ถ้าย้ายตำแหน่งปุ่มในภาพ ต้องแก้พิกัดด้วยเสมอ

พิกัดปุ่มอยู่ที่ richMenuMainObject_ ใน apps-script/08_RichMenu.js และไม่ได้ผูกกับภาพเลย ถ้าอัปโหลดภาพใหม่โดยไม่แก้พิกัด กดโลโก้จะกลายเป็นเปิดประกาศ กดประกาศจะเปิดคู่มือ สลับกันหมดทั้ง 60 คน โดยไม่มี error อะไรขึ้นมาเลย ไม่มีเทสต์ตัวไหนจับได้ ต้องเปิดไลน์กดดูเองทุกปุ่ม

ภาพดึงจาก GitHub Pages โดยตรง ไม่ต้องตั้ง Drive File ID ที่ละไฟล์อีกแล้ว เปลี่ยนที่เก็บได้ด้วย Script Property ชื่อ `RICHMENU_IMG_BASE_URL`

ข้อจำกัดของ LINE คือภาพต้องไม่เกิน 1 MB ต่อไฟล์ `build.py` จึงบันทึกเป็น JPEG คุณภาพ 92 ซึ่งได้ประมาณ 365 KB ถ้าภาพใหม่ใหญ่เกิน ให้ลดค่า `QUALITY` ใน `build.py`

ความปลอดภัย — ปิดแล้วอะไร ยังเปิดอยู่อะไร

ปิดไปแล้ว

Formula injection ในชีต

Google Sheets ถือว่าข้อความที่ขึ้นต้นด้วย `= + - @` คือสูตร ไม่ใช่ข้อความ ผู้โจมตีที่มีแค่บัญชี LINE (ไม่ต้องเป็นพนักงาน) จึงตั้งชื่อ LINE ตัวเองเป็นสูตร `IMPORTXML` ที่อ่านทั้งเว็บ `Employees` แล้วส่งออกไปเว็บของตัวเองได้ พอ HR เปิดชีต สูตรทำงานเองทันทีโดยไม่ต้องกดอะไร

ปิดด้วย `safeCell()` ใน `02_Data.js` ที่เติม `'` นำหน้าค่าที่ขึ้นต้นด้วยอักขระเหล่านั้นก่อนเขียนทุกครั้ง **ทุกการเขียนต้องผ่าน `appendRow()` หรือ `updateRow()` เท่านั้น** ห้ามเรียก `setValue()` ใส่ข้อมูลจากผู้ใช้โดยตรง เพราะจะเปิดช่องโหว่นี้กลับมา

ขอบเขตสิทธิ์ Google Drive

เดิมใช้ `drive.readonly` ซึ่งให้สิทธิ์อ่าน Drive **ทั้งบัญชี** ของเจ้าของ บน Web App ที่เปิดให้เข้าถึงแบบไม่ระบุตัวตน ตอนนี้แคบลงเหลือ `drive.file` ซึ่งเข้าถึงได้เฉพาะไฟล์ที่สคริปต์สร้างเอง

ยังเปิดอยู่ — ต้องตัดสินใจ

การตรวจลายเซ็น webhook ไม่เคยทำงานเลย

โค้ดใน `05_Webhook.js` อ่านลายเซ็นจาก query string แต่ LINE ส่งมาใน HTTP header `x-line-signature` ซึ่ง `doPost` ของ Apps Script อ่านไม่ได้ ตัวแปรจึงเป็น `null` เสมอ และเงื่อนไขตรวจสอบไม่เคยทำงาน

ผลคือใครที่รู้ URL `/exec` (ซึ่งอยู่ใน `config.js` ที่เสิร์ฟให้ทุกเบราว์เซอร์) สามารถปลอมเหตุการณ์ได้ เขาจะไม่ได้ข้อมูลกลับไป แต่ส่งล้างจุดแดงของพนักงานคนอื่น และเขียนแถวขยะลง `AuditLog` ได้ไม่จำกัด

วิธีแก้: วาง Cloudflare Worker (ประมาณ 20 บสสกด ฟรี) คั่นหน้า `/exec` ให้ตรวจลายเซ็นแล้วส่งต่อ โดยแนบลายเซ็นมาเป็น `?sig=` ซึ่งเป็นสิ่งที่โค้ดปัจจุบันรออยู่แล้ว — ท่อต่อไว้ครบแล้ว ขาดแค่ตัว proxy

สิ่งที่ออกแบบไว้ดีแล้ว อย่าไปรี้อ

- ทุกคำขอ API ตรวจ ID token กับ LINE ที่เดียวก่อนเข้า switch จึงไม่มี endpoint ไหนลืมตรวจได้
- ตัวตนถูกดึงจาก token ฝั่งเซิร์ฟเวอร์เสมอ ไม่เคยเชื่อ `empCode` ที่ส่งมาจาก client
- `publicEmp_()` คัดเฉพาะฟิลด์ที่อนุญาต และเขียนคอมเมนต์ระบุชัดว่าไม่ส่งอะไรบ้าง
- ผังองค์กรตัดสถานะพนักงานทั้ง 2 ชั้น เพื่อไม่ให้คนอื่นรู้ว่าเพื่อนถูกพักงาน
- หน้ายืนยันตัวตนตอบข้อความผิดพลาดเหมือนกันหมดทุกกรณี เพื่อไม่ให้ใช้เดารายชื่อพนักงานได้ และไม่รวมคนที่ลาออกแล้ว เพื่อไม่ให้ยืนยันได้ว่าใครเคยทำงานที่นี่

- รายงานที่ต้องใช้สิทธิ์ ตรวจสอบ role ซ้ำในโค้ดอีกชั้น ไม่เชื่อคอลัมน์ `audience` ในซีตอย่างเดียว เพราะ HR แก้ซีตนั้นได้ และการพิมพ์ผิดไม่ควรกลายเป็นข้อมูลรั่ว

สำรองข้อมูล และกู้คืนลงเครื่อง

ประวัติเวอร์ชันของ Google Sheets ไม่ใช้การสำรองข้อมูล

มันอยู่ในไฟล์เดียวกันและหายไปพร้อมกับบัญชี ถ้าบัญชีถูกระงับ ถูกลบ หรือคนที่ถือบัญชีลาออก ข้อมูลหายทั้งหมดพร้อมกัน ในขณะที่กฎหมายแรงงานไทยกำหนดให้เก็บทะเบียนลูกจ้างอย่างน้อย 2 ปี

สามขั้นที่ควรมี

ขั้น	อะไร	ความถี่ / สถานะ
1	สำเนาใน Google Drive	อัตโนมัติทุกอาทิตย์ 03:00 เก็บ 8 ชุด — ทำแล้ว
2	ดาวน์โหลดลงเครื่อง	ทุกไตรมาส ทำมือ — ดูขั้นตอนด้านล่าง
3	เจ้าของบัญชีสำรอง	ยังไม่ได้ทำ — ดูคำเตือนท้ายหัวข้อ

ขั้นที่ 1 — สำเนาอัตโนมัติใน Drive

ฟังก์ชัน `backupDatabase()` ใน `apps-script/12_Backup.js` คัดลอกทั้งไฟล์ไปยังโฟลเดอร์ สำรองฐานข้อมูล HR Hub ทุกอาทิตย์ เก็บไว้ 8 ชุด (ประมาณ 2 เดือน) ชุดที่เกินจะถูกทิ้งลงถังขยะ ไม่ลบถาวร จึงกู้ได้อีก 30 วัน

สั่งทำเดี๋ยวนี้ได้จากเมนู [สำรองข้อมูล → สำรองข้อมูลเดี๋ยวนี้](#)

ขั้นที่ 2 — ดาวน์โหลดลงเครื่อง (ทำทุกไตรมาส)

- 1 เปิดซีตต้นฉบับ
- 2 เมนู [ไฟล์ → ดาวน์โหลด → Microsoft Excel \(.xlsx\)](#)
- 3 เปลี่ยนชื่อไฟล์เป็น `HRHub_2569-Q3.xlsx` แล้วเก็บไว้ในฮาร์ดดิสก์ หรือไดรฟ์ภายนอกที่**ไม่ได้ซิงก์กับบัญชี Google** เดิม
- 4 เปิดไฟล์ที่ดาวน์โหลดมาดูสักครั้งว่าอ่านได้จริง — สำเนาที่เปิดไม่ได้ไม่ใช่สำเนา

ทำไมขั้นที่ 2 จำเป็นถึงที่มีขั้นที่ 1 แล้ว

สำเนาในขั้นที่ 1 ยังอยู่ใน**บัญชี Google** เดียวกันกับต้นฉบับ ถ้าบัญชีมีปัญหา สำเนาที่หายไปพร้อมกันทั้งหมด สำเนาที่อยู่นอกบัญชีเท่านั้นที่ป้องกันกรณีนี้ได้

ทำให้ขั้นที่ 2 อัตโนมัติ (ถ้าอยาก)

ใช้ `rclone` ซึ่งก็โพลเดอร์สำรองลงเครื่องได้

```
brew install rclone
rclone config # เลือก Google Drive แล้วตั้งชื่อ remote ว่า gdrive

rclone sync "gdrive:สำรองฐานข้อมูล HR Hub" ~/Backups/kohnaisoi-hrhub \
--drive-export-formats xlsx --progress
```

แล้วตั้งให้รันเองทุกสัปดาห์ด้วย `launchd` หรือ `cron`

กู้คืนข้อมูล

- 1 เปิดสำเนาที่ต้องการใน Drive แล้วเลือก **ไฟล์ → สร้างสำเนา**
- 2 คัดลอก ID ของไฟล์ใหม่จาก URL
- 3 เปิด Apps Script →  Project Settings → Script Properties แล้วเปลี่ยนค่า `SPREADSHEET_ID` ให้ใช้ไฟล์ใหม่
- 4 รัน `clearAllTableCache()` เพื่อล้างแคชของไฟล์เก่า
- 5 เปิดชีตใหม่แล้วกด **② ตรวจสอบการตั้งค่า** ว่าครบทุกรายการ
- 6 ทดสอบเปิดหน้าเว็บจากไลน์จริงหนึ่งครั้งก่อนประกาศให้พนักงานใช้

ความเสี่ยงที่ยังไม่มีใครแก้ได้นอกจากเจ้าของ

ทั้งซีต สคริปต์ ทริกเกอร์ โควตาอีเมล และโพลเดอร์สำรอง อยู่ใน**บัญชี Gmail ส่วนบุคคลบัญชีเดียวทั้งหมด** บัญชีส่วนบุคคลไม่มีผู้ดูแลระดับองค์กร ไม่มีผู้ติดต่อสำรอง ถ้าบัญชีถูกระงับหรือคนที่ถือลาออก ร้านจะเข้าถึงข้อมูลไม่ได้อีกเลย

ควรทำอย่างน้อยหนึ่งข้อ — เพิ่มบัญชีที่สองเป็นเจ้าของร่วมของทั้งซีตและ Apps Script project, ย้ายไปใช้ Google Workspace ของร้านซึ่งบัญชีได้จากผู้ดูแล, หรือตั้งผู้ติดต่อสำรองในบัญชี Google

รับมือเหตุขัดข้อง

พนักงานทุกคนเปิดหน้าเว็บไม่ได้

- 1 กดสอบ URL หลังบ้านก่อน — เปิดชิตแล้วกด  **ตรวจสอบ Webhook URL**
- 2 ถ้าขึ้น error เรื่องสิทธิ์ แปลว่า oauth scope เปลี่ยนแล้วยังไม่ได้ authorize ให้เปิด Apps Script editor รัน ฟังก์ชันอะไรก็ได้หนึ่งครั้งแล้วกดยอมรับ
- 3 ถ้ายังไม่หาย ให้ย้อนหลังบ้านกลับเวอร์ชันก่อนหน้า `./scripts/rollback.sh` — ใช้เวลาไม่กี่วินาทีและ URL ไม่เปลี่ยน
- 4 ดู execution log ใน Apps Script editor เพื่อหาสาเหตุที่แท้จริงทีหลัง

กดปุ่มในเมนูไลน์แล้วไปผิดหน้า

แปลว่าอัปเดตภาพเมนูใหม่โดยไม่ได้แก้ฟังก์ชันใน `richMenuMainObject_` ให้แก้ฟังก์ชันให้ตรงกับภาพ แล้วกดสร้างเมนูใหม่ทั้งหมด

จุดแดงไม่ขึ้น ทั้งที่มีประกาศใหม่

- 1 เปิดชิตกด  **Rich Menu → ดูรายการ Rich Menu ปัจจุบัน** ตรวจสอบว่ามีครบทั้ง 5 ชุดและ ID ถูกบันทึกไว้
- 2 ถ้าขาดชุดใดชุดหนึ่ง การติดจุดแดงจะไม่ทำงานแบบเจียบ ๆ ให้กด **สร้าง/อัปเดต Rich Menu ทั้งหมด**
- 3 ถ้ายังไม่ขึ้นเฉพาะบางคน ให้กด  **ล้างจุดแดงทุกคน** แล้วลองใหม่

แก้ไขชิตแล้วหน้าเว็บยังเป็นข้อมูลเก่า

ทริกเกอร์ล้างแคชไม่ได้ติดตั้ง กด  **ติดตั้งงานอัตโนมัติ** ใหม่ หรือรัน `clearAllTableCache()` เพื่อล้างทันที

ตารางกะที่หัวหน้าแก้หายไป

เป็นพฤติกรรมของ `generateScheduleFromPattern()` ซึ่งเก็บแถวไว้เฉพาะเมื่อคอลัมน์ `status` เป็นคำว่า `manual` ตัวพิมพ์เล็กเป๊ะ ๆ เท่านั้น แถวอื่นในช่วงที่สร้างใหม่จะถูกลบและสร้างจาก `ShiftPattern` ทั้งหมด

ถ้าได้จาก **ไฟล์ → ประวัติเวอร์ชัน** ในชิต ถ้าเพิ่งเกิด หรือจากไฟล์สำรองรายสัปดาห์

ระบบเจียบไปทั้งเดือน ไม่มีใครได้รับแจ้งเตือน

ตรวจไควตาข้อความก่อนที่  **สรุปการใช้งาน / ไควตาข้อความ** ถ้าใช้หมด 300 แล้วแปลว่ามีโค้ดที่ส่ง push แอบอยู่ ให้ค้นหาคำว่า `push` ในไฟล์เดอร์ `apps-script/` แล้วเอาออก

โควตาและข้อจำกัดที่ต้องจำ

ข้อจำกัด	ค่า	ผลถ้าชน
ข้อความ push ของ LINE (แพ็กเกจฟรี)	300 / เดือน	ส่งไม่ได้เลยและชื่อเพิ่มไม่ได้
ขนาดภาพ Rich Menu	1 MB / ไฟล์	LINE ปฏิเสธการอัปโหลด
เวลาทำงานของ Apps Script ต่อครั้ง	6 นาที	งานถูกตัดกลางคัน ข้อมูลอาจค้างครึ่ง ๆ
CacheService ต่อคีย์	100 KB	เก็บไม่ได้ จึงต้องแบ่งเป็นชิ้น
CacheService อายุสูงสุด	6 ชั่วโมง	แคชหมดอายุแล้วอ่านใหม่
PropertiesService ต่อค่า	9 KB	เก็บ JSON ก้อนใหญ่ไม่ได้
อีเมลจาก MailApp ต่อวัน	100 ฉบับ	อีเมลแจ้ง HR หยุดส่ง
เซลล์ทั้งหมดใน Google Sheet	10 ล้าน	เขียนเพิ่มไม่ได้ — ยังห่างมาก
ความยาวข้อความในเซลล์	50,000 ตัวอักษร	ข้อความถูกตัด

เวลาตอบสนองที่วัดได้จริง

ปลายทาง	เวลา
Apps Script <code>/exec</code>	~1,900 ms (มี redirect 1 ครั้ง = 2 การเชื่อมต่อ)
GitHub Pages (CDN)	~33 ms เมื่ออุ่นแล้ว, ~590 ms ครั้งแรก

ต่างกันประมาณ **50 เท่า** นี่คือเหตุผลที่ข้อมูลซึ่งไม่ค่อยเปลี่ยน อย่างคู่มือ คำถามที่พบบ่อย และคู่มือแอป ควรถูกเสิร์ฟจาก GitHub Pages แทนที่จะเรียกผ่าน Apps Script ทุกครั้ง

เอกสารนี้อธิบายระบบ ณ สิงหาคม 2569 เมื่อแก้ไขระบบแล้วกรุณาปรับคู่มือด้วย ไฟล์ต้นฉบับอยู่ที่ `manual/src/03-developer.html` สร้าง PDF ใหม่ด้วย `.venv-pdf/bin/python manual/build.py`